

It's About Time

How we defend with the clock, and attackers exploit it

Vladimir Atanasov

Security Folk

OWASP Sofia



Three lenses on one thing

01

COSMIC

What time actually is

02

ENGINEERING

How we captured it

03

SECURITY

Shield and weapon

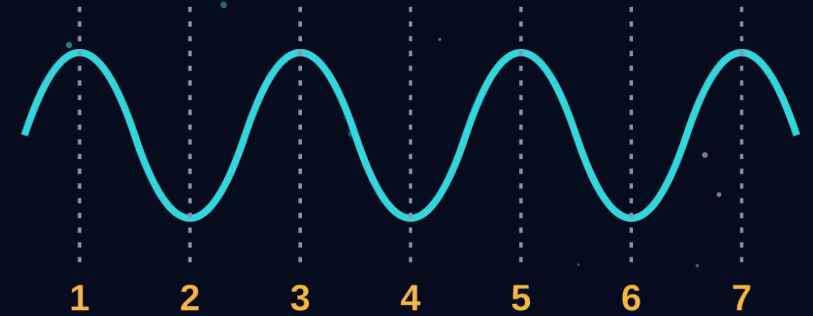
this is where we spend most of our time

**The universe never promised
that time would be constant.**

**We put a number on it anyway
and built everything on it.**

**A clock is just
something that changes,
regularly.**

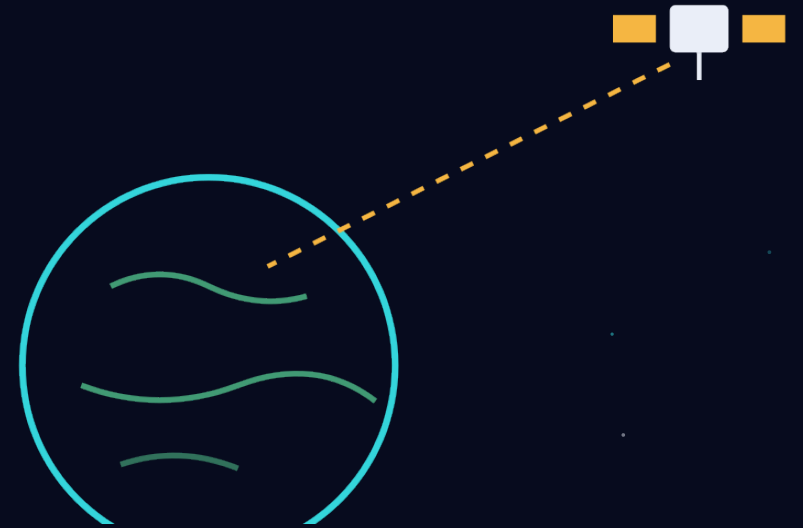
pendulum, quartz, atom: same trick, different speeds



COSMIC / PROOF TIME RUNS DIFFERENTLY

GPS is a clock measurement.

signal travel time, **then** distance, **then** position



Two effects, pulling opposite ways

~20,000 km up

general relativity



GRAVITY weaker up there

clock ticks faster

special relativity

SPEED ~14,000 km/h

clock ticks slower

The maths

+45

μs/day

gravity (faster)

-7

μs/day

speed (slower)

+38

μs/day

net, faster than ground

$$45 - 7 = 38$$

COSMIC / GPS

$38 \mu\text{s} \times \text{speed of light}$

$\approx 10 \text{ km}$

of position error. Every. Single. Day.

COSMIC TO ENGINEERING

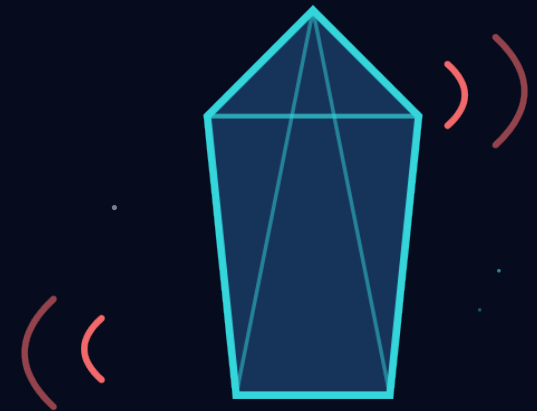
**Engineers built the correction in
on purpose.**

The time we trust is never automatically correct. Someone has to make it correct.

Quartz is always lying

- vibrates 32,768× / second
- temperature shifts the rate
- every crystal cut slightly differently
- ~15 to 30 s drift / month

the entire reason NTP has to exist



9,192,631,770

oscillations of a cesium-133 atom = one second

not a measurement. since 1967, this IS the definition

drifts ~1 second every few million years



Mistuned on purpose

- GPS satellites carry rubidium + cesium atomic clocks
- multiple per satellite: one active, rest as backup
- set to tick “wrong” before launch, so relativity makes them right

They were so sure time runs differently up there, they baked the error in. And they were right to the microsecond.

One number to agree on

UNIX time

seconds since 1 Jan 1970, 00:00 UTC

an arbitrary “year zero.” one ever-increasing integer that almost every computer trusts to mean “now”

Like a stopwatch the whole computing world started together, and agreed never to reset.

ENGINEERING / THE FOUNDATION HAS A DEADLINE

We built a clock that outlives the planet

32-bit **overflows 19 Jan 2038**

64-bit **overflows ~292 billion AD**

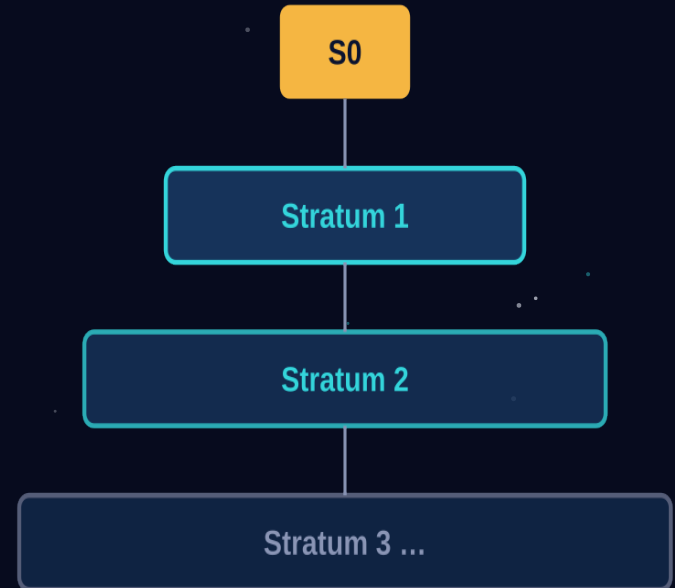
Earth's deadline **~1 billion years**

the sun brightens, oceans boil, long before the integer overflows



NTP, the committee

- every machine's clock drifts, so sync over the network
- a hierarchy of strata down to atomic / GPS references
- typically accurate to milliseconds



everyone roughly agrees, and almost nobody authenticates it

Proving the time is real

- classic NTP: UDP port 123, unauthenticated since 1985
- NTS adds a TLS handshake to bootstrap trust
- then authenticates the normal UDP time exchange

Time isn't secret. It's the same "now" for everyone. So NTS authenticates it, it does not encrypt it.



Public NTS servers

`time.cloudflare.com`

Cloudflare, anycast, global

`nts.netnod.se`

Netnod, Sweden, Stratum 1

`ptbtime1.ptb.de`

PTB, Germany, national time

chrony skips a source if NTS can't be established. Better no source than an unauthenticated one.

The delay attack

- an on-path attacker slows packets one direction
- NTP misreads the round trip, your clock drifts
- no forgery, so authentication can't stop it



Defend: multiple independent sources cross-checked, reject sudden offset jumps, and NTS does not fix this

PTP, to the nanosecond

NTP

milliseconds

everyone within a heartbeat

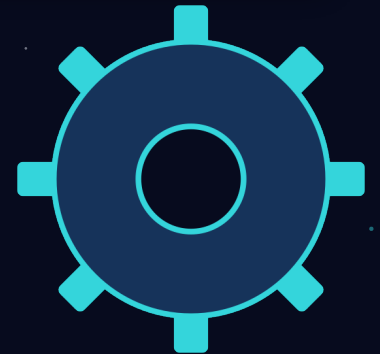
PTP

sub-microsecond

everyone to the same nanosecond

exchanges, 5G, industrial control, broadcast, where a microsecond costs millions

PTP often just trusts the network segment it runs on, which makes it an OT security surface.



All of these trust “now”

Kerberos

fails past ~5 min skew

TLS / certs

valid-from / valid-to dates

TOTP / SecurID

the code IS the time

JWT

exp / nbf claims

DNSSEC

signature validity windows

Kafka / DBs

event ordering, last-write-wins

**292 billion years of headroom
on the integer...**

**...yet Kerberos dies
at 5 minutes of skew.**

The threat was never running out of numbers. It's trusting the number we have.

03

Time as shield and weapon

how we defend with the clock, and how attackers turn it around

A forgotten door

- public RabbitMQ admin interface
- weak credentials, found in a routine assessment
- the messaging backbone for 50+ services

The scariest thing to expose is the thing everything depends on.



Rotate the credentials

the fast way to shut the door on the outside

Sometimes they worked.
Sometimes they didn't.

intermittent is the tell

The certificate was fine.
The clock was wrong.

- nodes authenticate each other over TLS
- cert validity is checked against the node's own clock
- a wrong clock rejects a valid cert

We changed the setting. **We had not enforced it.**

- the fix replicated to some nodes, not all
- for a window we did not know about, the old credentials still worked

Time skew handed us a false “remediation complete.”

The real fix had three layers

1

Rotate

new credentials

2

Upgrade

patch the old broker

3

Sync time

proper NTP on the nodes

The headline was credentials. Time discipline is what made it real.

Then the real work began

- 50+ services, several language stacks
- stale connections forced to reconnect
- client libraries incompatible with the new broker

One credential change, fifty blast radii.



What this one teaches

Changing a control is not enforcing it

and time sometimes is the gap between the two

Clock sync is silently load bearing

for security guarantees in distributed systems

The quiet dependency in the middle

is the one nobody audits until it breaks

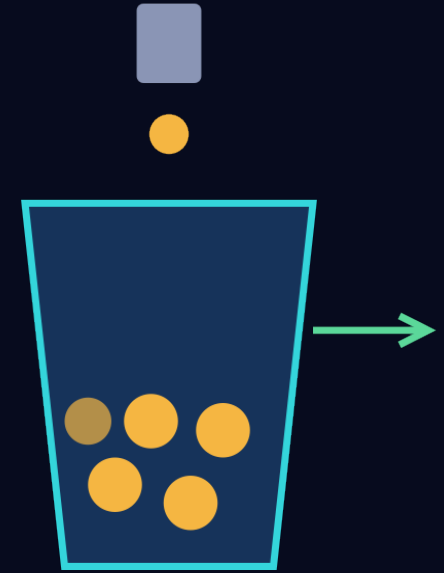
Small window, big payoff

- brute force and credential stuffing on login
- email enumeration on signup and account deletion
- the attacker needs only seconds

In a tiny window, huge benefit. That asymmetry is the whole game.

Rate limiting buys time

- catch the abuser by a signal: IP, user, user-agent
- time out further requests
- you don't block the attack, you make it too slow to be worth it



You spend their money

- abuse is rented by the hour: DDoS, stuffing, scraping
- a one-hour timeout outlasts what they paid for
- and buys you time to analyze and respond

Slow the attacker. Speed the defender.

IP isn't enough

- same user from two IPs slips an IP-based limit
- so you add user-based limiting
- different signals catch different abuse

Defenses evolve as attackers adapt.

Racing the counter

- check the count, then increment: a gap
- two requests both read “under the limit,” both pass
- the defense can contain the very race it prevents



This is a race condition at the heart of rate limiting.

Closing the gap

- token bucket on DynamoDB
- precise, but it needs a distributed lock
- the lock keeps the counter consistent across instances

github.com/mennanov/limiters

One coupon, two checkouts

- fire two purchases in parallel on one discount code
- both validate the code before either commits
- same race, this time in application logic

Seen in the wild. Same shape as the counter race.

What this one teaches

Rate limiting is time asymmetry as a defense

slow the attacker, speed the defender

Check-then-act is the recurring flaw

make it atomic, or it gets raced

Different signals catch different abuse

IP, user, and beyond, layered

THE CLOSE

Attackers have unlimited time.
Defenders race the clock.

They only need to win once, whenever. You have to win every time, continuously.

Take it home

Make check-and-use atomic

locks or atomic ops, not check-then-act

Treat time sync as security-critical

NTP and NTS are attack surface, not plumbing

Audit every trusted timestamp

certs, tokens, logs, replication

Time is like money.

*The universe never fixed its value. Engineers agreed on a currency.
Our whole job is stopping people from counterfeiting it.*

It's about time we treated it that way.

Questions?

